Bitácora del proyecto

proyectoCSV - App de clases y reservas (Expo / React Native)

Resumen de consultas y soluciones - 12 de octubre de 2026

#	Tema
1	Navegación: dónde debe vivir la lógica
2	Barra inferior: Bottom Tabs
3	Error al navegar a ClasesStack
4	Entendiendo el Stack Navigator
5	Iconos Ionicons (reserva y bienvenida)
6	Reservas con validación de horario
7	Qué es createContext
8	Errores en ReservasScreen
9	Pantalla duplicada (dos menús)
10	Cupos que se reinician
11	Formularios en React Native
12	Input con botones para escoger
13	Teclado que bloquea el botón
14	Mostrar datos del usuario tras registrarse
15	Pendientes y mejoras sugeridas

1. Navegación: dónde debe vivir la lógica

> ***Lo que pediste:*** *InicioScreen tiene un menú con Pressable y existe una carpeta navigation con ClasesStack. ¿Se navega desde la pantalla o desde el archivo de navegación?*

Solución: separar responsabilidades. La estructura (qué rutas existen) vive en `navigation/`; la pantalla solo llama a `navigation.navigate('NombreRuta')` sin importar ClasesStack.

```
// screens/InicioScreen.js
const navigation = useNavigation();
<Pressable onPress={() => navigation.navigate('Clases')}>
  <Text>Clases</Text>
</Pressable>
```

2. Barra inferior: Bottom Tabs

> ***Lo que pediste:*** *El menú es una barra fija en la parte inferior.*

Solución: usar `@react-navigation/bottom-tabs` en vez de armar la barra con Pressable. Resuelve barra fija, estado activo e iconos.

```
npm install @react-navigation/bottom-tabs

// navigation/TabNavigator.js
const Tab = createBottomTabNavigator();
<Tab.Navigator screenOptions={{ headerShown: false }}>
  <Tab.Screen name="Inicio" component={InicioScreen} />
  <Tab.Screen name="Clases" component={ClasesStack} />
  <Tab.Screen name="Reservas" component={ReservasScreen} />
</Tab.Navigator>

// App.js: un solo NavigationContainer
<NavigationContainer><TabNavigator /></NavigationContainer>
```

- InicioScreen pasa a ser solo contenido (sin menú).
- Cada tab con su propio stack conserva su historial.
- Iconos por tab con `tabBarIcon` y `focused ? 'home' : 'home-outline'`.

3. Error al navegar a ClasesStack

> ***Lo que pediste:*** *Al navegar desde InicioScreen salía un error: no identificaba el archivo.*

Causa real: las demás pantallas del stack tenían el archivo vacío. Sin `export default`, React Navigation recibe undefined como componente.

- navigate() recibe **nombres de ruta (strings)**, no componentes.
- Un import no registra la ruta; eso lo hace Stack.Screen.
- Error 'was not handled by any navigator': nombre no registrado o mal escrito.

Placeholder mínimo para pantallas nuevas:

```
import { View, Text } from 'react-native';
export default function DetalleClaseScreen() {
  return <View><Text>DetalleClaseScreen</Text></View>;
}
```

4. Entendiendo el Stack Navigator

> ***Lo que pediste:*** *No entendía bien la parte del stack navigator.*

Idea: un navegador es una agenda de contactos. Cada Stack.Screen tiene un **nombre** y un **componente**; navigate('Clases') busca ese nombre en la agenda.

```
RootNavigator
 |-- "Inicio" -> InicioScreen
 '-- "Clases" -> ClasesStack
                   |-- "ListaClases"  -> ...
                   '-- "DetalleClase" -> ...
```

Checklist: (1) existe un Stack.Screen con ese nombre, (2) el nombre es idéntico, (3) InicioScreen está en ese navegador o uno padre, (4) App.js renderiza el navegador dentro de NavigationContainer.

5. Iconos Ionicons

> ***Lo que pediste:*** *Iconos para 'reserva' y para 'bienvenida / inicio'.*

Uso	Opciones (Ionicons)
Reserva	calendar, bookmark, ticket, time, today, checkmark-circle, receipt, clipboard
Bienvenida / Inicio	home, hand-left, happy, enter, sparkles, star, compass, grid

Recomendado: calendar para Reservas y home para Inicio. Cada nombre tiene variante `-outline`. Catálogo completo en ionic.io/ionicons.

6. Reservas con validación de horario

> ***Lo que pediste:*** *Guardar el horario al reservar, mostrar la reserva en ReservasScreen y validar que el estudiante no tenga otra clase a la misma hora y día, con alerta si hay conflicto.*

Solución: un Context (ReservasHechas) con la lista de reservas y una función `agregarReserva` que valida antes de guardar.

```
const agregarReserva = (clase, horario) => {
  const conflicto = reservas.some(
    (r) => horarioSolo(r.horario) === horarioSolo(horario)
  );
  if (conflicto)
    return { success: false, message: 'Ya tienes una reserva para este horario' };

  setReservas((prev) => [...prev,
    { id: `${clase.id}-${Date.now()}`, clase, horario }]);
  return { success: true };
};

// En DetalleClase: manejar también el caso de error
const resultado = agregarReserva(clase, horarioSeleccionado);
if (resultado.success) Alert.alert("Reserva realizada con éxito.");
else Alert.alert("No se puede reservar", resultado.message);
```

- La validación compara solo el texto del horario: si no incluye el día, dos clases en días distintos a la misma hora se bloquearían entre sí.
- Si las clases tienen duración, validar solapamiento con inicio y fin.
- Con backend, validar también en el servidor.

7. Qué es createContext

> ***Lo que pediste:*** *Para qué sirve createContext y cómo se usa.*

Comparte datos entre componentes sin pasarlos por props nivel a nivel. Tiene tres piezas:

```
const MiContext = createContext();                // 1. crear la caja

<MiContext.Provider value={{ datos, funciones }}>  // 2. proveer
  {children}
</MiContext.Provider>

const { datos } = useContext(MiContext);          // 3. consumir

export const useMiContexto = () => useContext(MiContext); // atajo
```

- Úsalo para datos que varias pantallas necesitan (usuario, reservas, carrito).
- Para estado local de una sola pantalla basta useState.
- Usar useContext fuera del Provider devuelve undefined: el Provider va en App.js.

8. Errores en ReservasScreen

> ***Lo que pediste:*** *Error al visualizar las reservas (se pegaron DetalleClase, ReservasHechas y ReservasScreen).*

Diagnóstico:

- **Crash:** `<text>` en minúscula en lugar de `<Text>`. Solo fallaba con reservas porque está dentro de renderItem.
- Props mal escritas en FlatList: `contenido` -> `contentContainerStyle`; `ReservasSeparador` -> `ItemSeparatorComponent`.
- Estilos usados que no existían en StyleSheet (profesor, horario, detalle vs. detalleReserva, horarioReserva, vacio).
- Faltaba exportar `useReservas` en ReservasHechas.
- En DetalleClase faltaba el else para mostrar la alerta de conflicto.

```
export const useReservas = () => useContext(ReservasContext);

<FlatList
  data={reservas}
  keyExtractor={(r) => r.id}
  contentContainerStyle={[styles.lista, reservas.length === 0 && styles.listaVacia]}
  ItemSeparatorComponent={() => <View style={styles.separador} />}
  ...
/>
```

9. Pantalla duplicada (dos menús)

> ***Lo que pediste:*** *La pantalla se ve doble: el mismo contenido y dos menús, uno arriba al medio y otro abajo.*

Causa típica: navegadores anidados que renderizan lo mismo. Revisar, en orden:

- Un TabNavigator o navegador dentro de InicioScreen o ClasesStack.
- El menú viejo de Pressable sigue en InicioScreen junto a Bottom Tabs.

- Navegador registrado dos veces (en App.js y en RootNavigator).
- NavigationContainer repetido; debe haber uno solo en App.js.

Truco: buscar con Ctrl+Shift+F NavigationContainer, createBottomTabNavigator y <TabNavigator; cada uno debe usarse en un solo lugar.

```
App.js
 '-- NavigationContainer (uno solo)
      '-- TabNavigator
           |-- Inicio   -> InicioScreen (solo contenido)
           |-- Clases   -> ClasesStack
           '-- Reservas -> ReservasScreen
```

10. Cupos que se reinician

> ***Lo que pediste:*** *Reservo un cupo, cambio de pantalla, vuelvo a la clase y los cupos están otra vez completos.*

Causa: cupos vivía en un useState dentro de DetalleClase; al salir de la pantalla el componente se desmonta y el estado se pierde.

Solución: no guardar los cupos, calcularlos: cupos totales menos reservas de esa clase.

```
// En ReservasProvider
const cuposDisponibles = (clase) => {
  const reservadas = reservas.filter((r) => r.clase.id === clase.id).length;
  return Number(clase.cupos) - reservadas;
};
// value={{ reservas, agregarReserva, cuposDisponibles }}

// En DetalleClase (se elimina useState de cupos y setCupos)
const { agregarReserva, cuposDisponibles } = useReservas();
const cupos = cuposDisponibles(clase);
```

También conviene validar cupos dentro de agregarReserva para que ninguna pantalla pueda saltarse la regla.

11. Formularios en React Native

> ***Lo que pediste:*** *Formulario en React.*

En React Native no existe `<form>`: se usan TextInput controlados, useState y un Pressable como botón de envío.

```
const [form, setForm] = useState({ nombre: '', email: '' });
const handleChange = (campo, valor) =>
  setForm((prev) => ({ ...prev, [campo]: valor }));

<TextInput
  value={form.nombre}
  onChangeText={(t) => handleChange('nombre', t)}
  placeholder="Tu nombre"
/>
<TextInput
  value={form.email}
  onChangeText={(t) => handleChange('email', t)}
  keyboardType="email-address"
  autoCapitalize="none"
/>
<Pressable onPress={handleEnviar}><Text>Enviar</Text></Pressable>
```

- Validar antes de enviar y mostrar errores por campo.

- Props útiles: keyboardType, secureTextEntry, autoCapitalize, multiline, onSubmitEditing.
- Para formularios grandes existe react-hook-form.

12. Input con botones para escoger

Lo que pediste: *Un input con botón interactivo para que el usuario escoja.*

- **Selección única (chips):** mismo patrón de los horarios; estado con el valor activo.
- **Selección múltiple:** estado como arreglo y alternar con includes / filter.
- **Dropdown:** un Pressable con aspecto de input que abre un Modal con FlatList de opciones.
- **Otros:** Switch (sí/no), @react-native-community/datetimepicker (fecha/hora), @react-native-picker/picker (selector nativo).
- Pocas opciones (hasta unas 5): botones visibles. Muchas: dropdown.

```
const [seleccion, setSeleccion] = useState([]);
const toggle = (op) => setSeleccion((prev) =>
  prev.includes(op) ? prev.filter((o) => o !== op) : [...prev, op]);
```

13. Teclado que bloquea el botón

Lo que pediste: *El teclado no deja presionar el botón de registro.*

Dos problemas distintos:

- **El teclado tapa el botón:** envolver con KeyboardAvoidingView y ScrollView.
- **El primer toque solo cierra el teclado:** `keyboardShouldPersistTaps="handled"` en el ScrollView.

```
<KeyboardAvoidingView style={{ flex: 1 }}
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}>
  <ScrollView keyboardShouldPersistTaps="handled"
    contentContainerStyle={{ flexGrow: 1, padding: 16 }}>
    {/* inputs y botón */}
  </ScrollView>
</KeyboardAvoidingView>

// Encadenar campos y enviar desde el teclado
<TextInput returnKeyType="next" blurOnSubmit={false} />
<TextInput returnKeyType="done" onSubmitEditing={handleRegistro} />
// Keyboard.dismiss() para cerrarlo por código
```

14. Mostrar datos del usuario tras registrarse

Lo que pediste: *Al registrarse, que ya no salga el formulario sino los datos del usuario registrado.*

Solución: guardar el usuario en un Context y usar renderizado condicional en la pantalla.

```
// components/UsuarioContext.js
const [usuario, setUsuario] = useState(null);   // null = no registrado
const registrar = (datos) => setUsuario(datos);
const cerrarSesion = () => setUsuario(null);

// App.js: <UsuarioProvider> envolviendo al navegador

// En la pantalla (los hooks van ANTES del if)
const { usuario, registrar, cerrarSesion } = useUsuario();
if (usuario) {
  return (<View>
```

```
    <Text>Nombre: {usuario.nombre}</Text>
    <Text>Email: {usuario.email}</Text>
    <Pressable onPress={cerrarSesion}><Text>Cerrar sesión</Text></Pressable>
  </View>);
}
return ( /* formulario */ );
```

- No guardar ni mostrar la contraseña en el estado del usuario.
- Se puede exigir registro antes de reservar comprobando if (!usuario).
- El estado está en memoria: se pierde al cerrar la app.

15. Pendientes y mejoras sugeridas

- **Persistencia con AsyncStorage** (reservas y usuario) si deben sobrevivir al cierre de la app: `npx expo install @react-native-async-storage/async-storage`.
- Permitir **cancelar reservas** (los cupos se recuperan solos al calcularlos).
- Incluir el **día** en cada horario o validar solapamiento con hora de inicio y fin.
- Unificar el formato de horarios para que la comparación sea consistente.
- Validar cupos dentro de agregarReserva, no solo en la pantalla.
- Vincular las reservas al usuario registrado.